JGi JAIN DEEMED-TO-BE UNIVERSITY | FACULTY OF ENGINEERING AND TECHNOLOGY

School of Computer Science and Engineering

(Computer Engineering Software Engineering)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

2025-2026

(VI Semester)

A Project Report on

AN EFFECTIVE SMART EVENT PLANNER AND COST MANAGEMENT SYSTEM

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER ENGINEERING
SOFTWARE ENGINEERING

Submitted by

GIRISH R V (23BTRSN022)

NILIN ROSE (23BTRSN034)

ANAMIKA H (23BTRSN012)

TANISH BALWAL L (23BTRSN048)

Under the guidance of

Dr Anthoniraj S

Professor

Department of Computer Engineering Software Engineering

School of Computer Science & Engineering

Faculty of Engineering & Technology

JAIN (Deemed to-be University)

Department of Computer Engineering Software Engineering

School of Computer Science & Engineering
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled **"AN EFFECTIVE SMART EVENT PLANNER AND COST MANAGEMENT SYSTEM"** is carried out by **GIRISH R V (23BTRSN022), NILIN ROSE (23BTRSN034), ANAMIKA H (23BTRSN012), TANISH BALWAL L (23BTRSN048)** Bonafide students of Bachelor of Technology at the School of Computer Science and Engineering , Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor Technology in Computer Engineering Software Engineering, during the year **2025-2026**.

Dr. Anthoniraj S
Project Guide ,
Computer Engineering, Software Engineering
School of Computer Science and Engineering
Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Dr. Santhosh S
Program Head,
Computer Engineering, Software Engineering
School of Computer Science and Engineering
Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Dr. Kamlesh Tiwari
Head of Department,
Computer Science & Engineering
Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Name of the Examiner — Signature of Examiner

1.

2.

DECLARATION

We , **G**IRISH **R V (23BTRSN022), N**ILIN **R**OSE **(23BTRSN034),** A**N**AMIKA **H (23BTRSN012),** T**ANISH B**ALWAL **L (23BTRSN048) students of 6TH semester B.Tech in Computer Engineering Software Engineering, at School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed to- be University), hereby declare that the Project work titled "AN EFFECTIVE SMART EVENT PLANNER AND COST MANAGEMENT SYSTEM" has been carried out by us and submitted in partial fulfilment for the award of degree in Bachelor of Technology in Computer Engineering Software Engineering during the academic year 2025-2026. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.**

GIRISH R V:
(23BTRSN022)

NILIN ROSE:
(23BTRSN034):

ANAMIKA H:
(23BTRSN012)

TANISH BALWAL L:
(23BTRSN048)

Place: Bangalore

Date:

ACKNOWLEDGEMENT

It is a great pleasure for me to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, I take this opportunity to express my sincere gratitude to Faculty of Engineering & Technology, JAIN (Deemed to-be University) for providing me with a great opportunity to pursue my Bachelor's Degree in this institution.

I am deeply thankful to several individuals whose invaluable contributions have made this project a reality. I wish to extend my heartfelt gratitude to ***Dr. Chenraj Roy Chand, Chancellor****, for his tireless commitment to fostering excellence in teaching and research at Jain (Deemed-to-be-University). I am also profoundly grateful to the honorable* ***Dr. Dinesh Nilkant, Pro Vice Chancellor****, for their unwavering support. Furthermore, I would like to express my sincere thanks to* ***Dr. Jitendra Kumar Mishra, Registrar****, whose guidance has imparted invaluable qualities and skills that will serve us well in our future endeavors.*

I extend my sincere gratitude to ***Dr. Kamlesh Tiwari, Head of Department*** *of the School of Computer Science & Engineering within the Faculty of Engineering & Technology, for their constant encouragement and expert advice. Additionally, I would like to express my appreciation to* ***Dr. Deepak Kumar Sinha, Deputy Director (Students & Industry Relations),*** *for their invaluable contributions and support throughout this project.*

It is a matter of immense pleasure to express my sincere thanks to ***Dr. Santhosh S, Program Head, Computer Engineering Software Engineering****, School of Computer Science & Engineering Faculty of Engineering & Technology for providing right academic guidance that made my task possible.*

I would like to thank our guide ***Dr. Anthoniraj S****,* ***Assistant Professor****,* ***Dept. of Computer Engineering Software Engineering****, for sparing his valuable time to extend help in every step of my work, which paved the way for smooth progress and fruitful culmination of the project.*

I would like to thank our ***Project Coordinator Dr. Ramesh Babu Jonnalagadda****, and all the staff members of Computer Engineering Software Engineering for their support.*

I am also grateful to my family and friends who provided me with every requirement throughout the course.

I would like to thank one and all who directly or indirectly helped me in completing the work successfully.

Signature of Student(s)

Girish R V
23BTRSN022

Nilin Rose
23BTRSN034

Anamika H
23BTRSN012

Tanish Balwal L
23BTRSN048

TABLE OF CONTENTS

LIST OF FIGURES

ABSTRACT

Event Management System (EventMS) is a comprehensive, web-based event management platform built with Next.js 16 and Supabase, designed to modernize the entire lifecycle of event planning, coordination, and attendee management. The system serves three distinct user roles — customers/organizers, vendors, and administrators — through dedicated dashboards and role-based access control enforced at the database level via Row Level Security (RLS).

A defining feature of this platform is the integration of five custom machine learning algorithms implemented natively in TypeScript. XSimGCL provides graph-contrastive-learning-based event recommendations for warm users (those with ≥3 interactions), while GNN-CF (Graph Neural Network – Cross-Domain Collaborative Filtering) handles cold-start scenarios for new users. The MOEA/D-DRA-NEF algorithm performs multi-objective budget optimization, returning Pareto-optimal vendor bundles labelled by cost-quality trade-offs. The iTransformer architecture delivers 7- and 14-day attendance forecasts with configurable confidence intervals, and GAT + K-Means clustering enables community detection across events and users.

The platform also features an AI-powered conversational assistant powered by Google Gemini (with Ollama/HuggingFace as a local fallback), a vendor marketplace with service request workflows, interactive Leaflet-based event maps, a real-time RSVP system with waitlist management, and a dedicated Admin Dashboard for algorithm evaluation, BPR training, and research data export.

By unifying intelligent recommendations, cost optimization, predictive analytics, and community-aware discovery within a single scalable platform, EventMS addresses the core limitations of fragmented event management tools and delivers a research-grade yet production-ready system suitable for real-world deployment.

Chapter 1

1.1 Introduction

Event management is a complex and dynamic process that involves careful coordination of multiple stakeholders, resources, and activities to ensure successful execution of events ranging from small gatherings to large-scale conferences and festivals. In today's fast-paced digital world, there is a growing demand for intelligent, integrated platforms that can streamline the entire event lifecycle — from planning and vendor management to attendee engagement and post-event analysis.

This project presents **EventMS** — An Effective Smart Event Planner and Cost Management System. It is a full-stack web application designed to address the inefficiencies of traditional event management tools by combining modern web technologies with advanced machine learning capabilities. The system provides a unified platform for event organizers, vendors, and administrators, offering real-time collaboration, intelligent recommendations, budget optimization, and predictive analytics.

1.2 Background & Motivation

Traditional event management relies heavily on fragmented tools such as spreadsheets, multiple messaging platforms, separate vendor portals, and basic booking websites. This leads to several challenges including poor communication, budget overruns, inefficient vendor selection, inaccurate attendance predictions, and limited personalization for attendees.

As the scale and frequency of events increase, organizers face mounting pressure to deliver exceptional experiences while controlling costs and managing complex logistics. There is a clear need for a comprehensive, intelligent system that can automate repetitive tasks, provide data-driven insights, and facilitate seamless collaboration among all stakeholders.

EventMS was developed to bridge these gaps. By leveraging modern technologies like Next.js 16, Supabase, and five custom-built machine learning algorithms, the platform transforms event planning from a reactive, manual process into a proactive, intelligent, and efficient experience. The motivation behind this project stems from the desire to create a production-ready system that not only solves real-world problems in event management but also contributes to academic research in applied machine learning for recommendation systems, optimization, and forecasting.

1.3 Objectives

The goal of this work is to build a user-friendly, interactive web-based platform that The primary objectives of this project are as follows:

- To design and develop a scalable, full-stack event management platform using Next.js 16 (App Router) and Supabase with strong emphasis on performance, security, and user experience.
- To implement secure role-based access control for three user types — Customers/Organizers, Vendors, and Administrators — using Supabase Auth and Row Level Security (RLS).
- To integrate five custom machine learning algorithms natively in TypeScript:
    - XSimGCL for personalized event recommendations (warm users)
    - GNN-CF for cold-start recommendations
    - MOEA/D-DRA-NEF for multi-objective budget optimization
    - iTransformer for attendance forecasting
    - GAT + K-Means for community detection
- To build a smart budget planner that generates Pareto-optimal vendor bundles based on cost-quality trade-offs.
- To develop an AI-powered conversational chatbot for real-time assistance to users.
- To create an interactive vendor marketplace with service request workflows and status tracking.
- To provide attendance forecasting and community-based discovery features to enhance decision-making.

- To follow a structured Project Centric Learning methodology including requirement analysis, system design, iterative development, testing, and deployment.

1.4 Benefits of Research

This project offers several significant benefits:

- **For Event Organizers**: Reduced planning time, optimized budgeting, intelligent vendor recommendations, accurate attendance forecasts, and streamlined coordination through a single platform.
- **For Vendors**: Increased visibility, easier access to business opportunities, simplified request management, and clear earnings tracking.
- **For Attendees/Users**: Personalized event recommendations, seamless RSVP experience, interactive maps, and AI assistance for better event discovery and participation.
- **Academic & Research Contribution**: Implementation and evaluation of five advanced ML algorithms (including novel applications of XSimGCL, MOEA/D-DRA-NEF, and iTransformer in event management) provides valuable insights into practical deployment of machine learning in real-world systems.
- **Industry Impact**: Delivers a production-ready, scalable solution that can be adopted by event management companies, colleges, corporates, and startups to improve efficiency and user satisfaction.
- **Technological Advancement**: Demonstrates the successful integration of modern full-stack development practices with cutting-edge AI/ML techniques in a single cohesive application.

The successful completion of this project showcases how intelligent systems can significantly enhance traditional domains like event management while contributing to both academic knowledge and practical industry solutions.

Chapter 2

2.1 LITERATURE SURVEY

Category	Year	Title & Authors	Key Contribution	Relevance to Proposed Work
Core Algorithm Papers	2024	XSimGCL: Towards Extremely Simple Graph Contrastive Learning for Recommendation (Yu et al.)	Simplified graph contrastive learning with strong performance.	Core graph-based contrastive method for recommendations.
Core Algorithm Papers	2024	iTransformer: Inverted Transformers Are Effective for Time Series Forecasting (Liu et al.)	Transformer variant focusing on variates for better forecasting.	Backbone for temporal/sequential event modeling.
Core Algorithm Papers	2025	A New Multi-objective Optimization Algorithm Based on Decomposition (Tan et al.)	Improved MOEA/D with dynamic resource allocation.	Supports multi-objective optimization (accuracy vs. diversity).
Core Algorithm Papers	2024	Community Detection Using Graph Attention Networks (Ippatapu & Bhadra)	GAT + clustering for community detection.	Used for user/event community modeling.
Foundational Recommender Systems	2020	LightGCN (He et al.)	Simplified yet powerful GCN for recommendation.	Baseline GNN recommender model.
Foundational Recommender Systems	2009	BPR: Bayesian Personalized Ranking (Rendle et al.)	Pairwise ranking for implicit feedback.	Core optimization objective.
Foundational Recommender Systems	2018	Sequential Recommendation via CNN (Tang & Wang)	Sequence-based recommendation modeling.	Handles user behavior sequences.
Graph Neural Networks	2017	GCN (Kipf & Welling)	Introduces Graph Convolutional Networks.	Foundation of all GNN components.
Graph Neural Networks	2018	GAT (Veličković et al.)	Attention mechanism in	Key for community detection & message

			graphs.	passing.
Transformer Architecture	2017	Attention Is All You Need (Vaswani et al.)	Introduces Transformer architecture.	Basis for iTransformer.
Multi-Objective Optimisation	2007	MOEA/D (Zhang & Li)	Decomposition-based multi-objective optimization.	Core framework for optimization module.
Multi-Objective Optimisation	2002	NSGA-II (Deb et al.)	Classic multi-objective genetic algorithm.	Baseline comparator.
Community Detection	2006	Modularity (Newman)	Defines modularity metric.	Evaluates community quality.
Event Recommendation Systems	2012	Event-Based Social Networks (Liu et al.)	Defines EBSN concept.	Establishes problem domain.
Evaluation	2002	NDCG (Järvelin & Kekäläinen)	Ranking evaluation metric.	Used for performance evaluation.

2.2 PROBLEM FORMULATION AND PROPOSED WORK

2.2.1 Introduction

Event management is a multifaceted process that requires careful coordination among organizers, vendors, attendees, and various service providers. While numerous event management tools exist in the market, most suffer from critical shortcomings such as lack of integration, absence of intelligent decision support, poor scalability, and limited use of modern technologies. This results in increased manual effort, higher operational costs, budget overruns, inefficient vendor selection, and suboptimal attendee experiences.

To address these challenges, this project proposes **EventMS** — An Effective Smart Event Planner and Cost Management System. The proposed system aims to create a unified, intelligent, and scalable platform that leverages modern web technologies and advanced machine learning algorithms to streamline the entire event lifecycle. This section presents the problem formulation, highlights the limitations of existing systems, and outlines the proposed solution, system architecture, and implementation approach.

2.2.2 Problem Statement:

Traditional event management systems and tools face several significant limitations:

- **Fragmented Workflows**: Organizers are forced to use multiple disconnected platforms for event creation, vendor management, attendee registration, budgeting, and communication, leading to data inconsistency and increased workload.

- **Lack of Intelligent Support**: Most platforms do not provide personalized event recommendations, smart budget optimization, or predictive analytics for attendance, resulting in poor decision-making.

- **Inefficient Vendor Management**: Finding suitable vendors, comparing quotes, and tracking service requests is largely manual and time-consuming.
- **Limited Personalization and Discovery**: Attendees struggle to discover relevant events due to the absence of intelligent recommendation engines and community-based filtering.
- **Budget Overruns**: Without proper optimization tools, organizers often exceed budgets due to suboptimal vendor selection.
- **Poor Scalability and Real-time Capabilities**: Existing systems frequently lack real-time features such as live RSVP updates, interactive maps, and dynamic notifications.
- **Absence of Role-Based Intelligence**: Different stakeholders (organizers, vendors, and admins) do not have tailored dashboards with relevant insights and tools.

These problems lead to inefficient event planning, higher costs, lower attendee satisfaction, and missed opportunities for data-driven improvements. There is a pressing need for a comprehensive, AI-powered platform that integrates all aspects of event management into a single intelligent system.

2.2.3 System Architecture / Model

The proposed **EventMS** follows a modern full-stack architecture with clear separation of concerns:

Frontend Layer: Built using Next.js 16 (App Router) and React 19 with TypeScript. It provides responsive, interactive user interfaces for three distinct roles — Customer/Organizer, Vendor, and Admin — with role-based routing and dynamic dashboards.

Backend & Database Layer: Powered by Supabase (PostgreSQL) with Row Level Security (RLS) for secure data access. It handles authentication (JWT), real-time subscriptions, file storage (images, event galleries), and API routes.

Machine Learning Layer: Five custom algorithms implemented in TypeScript:

- Recommendation Engine (XSimGCL + GNN-CF)
- Budget Optimization Engine (MOEA/D-DRA-NEF)
- Attendance Forecasting (iTransformer)
- Community Detection (GAT + K-Means)

AI Assistant Layer: Integrated conversational chatbot powered by Google Gemini with local fallback support.

External Integrations: Leaflet maps for venue visualization, geocoding services, and real-time notification systems.

The architecture is designed to be scalable, secure, and edge-compatible, with caching mechanisms at both application and algorithm levels for optimal performance. All sensitive operations are validated using Zod schemas, and data fetching is optimized using SWR for efficient client-side state management.

2.2.4 Proposed Algorithms

EventMS integrates **five custom machine learning algorithms** implemented natively in TypeScript. These algorithms form the intelligent core of the platform and significantly differentiate it from conventional event management systems.

1. Recommendation Engine

- **XSimGCL (for Warm Users)**: A Graph Contrastive Learning-based model used for users with three or more interactions. It learns rich user-event representations through graph augmentation and contrastive learning to deliver highly personalized event recommendations.

- **GNN-CF (Graph Neural Network – Cross-Domain Collaborative Filtering)**: Designed specifically for cold-start users (new users with fewer than 3 interactions). It leverages cross-domain knowledge to provide accurate recommendations even with limited user history.

2. Budget Optimizer – MOEA/D-DRA-NEF

- A Multi-Objective Evolutionary Algorithm based on Decomposition with Dynamic Resource Allocation and Neighbourhood Evolution Framework.

- Given an event budget and required service categories, it generates a set of Pareto-optimal vendor bundles. These bundles are categorized into **Budget Pick**, **Best Value**, and **Premium** options, helping organizers make informed trade-offs between cost and quality.

3. Attendance Forecasting – iTransformer

- An advanced time-series forecasting architecture based on inverted Transformers.

- Predicts expected attendance for events 7 and 14 days in advance with configurable confidence intervals.

- The forecasts help organizers with better capacity planning, resource allocation, and marketing decisions.

4. Community Detection – GAT + K-Means

- Combines Graph Attention Networks (GAT) with K-Means clustering.

- Identifies interest-based communities among users and events.

- This enables powerful community-aware filtering and discovery, allowing users to explore events that match their preferences and social circles.

5. AI Conversational Assistant

- Powered by Google Gemini (gemini-3-flash-preview) with Ollama/Hugging Face local fallback.
- Provides context-aware responses for event-related queries, planning assistance, vendor suggestions, and troubleshooting.

All algorithms include:

- Efficient caching mechanisms (30–60 minutes) stored in the algorithm results table.
- Proper error handling and fallback strategies.
- Explainability features (where applicable) shown in the user interface.
- Performance monitoring and evaluation tools available in the Admin Dashboard.

These algorithms work together seamlessly to create a truly smart and adaptive event management ecosystem.

2.2.5 Proposed Work

The proposed work is a roadmap of development, testing, and the development of EventMS follows a structured six-phase approach:

Phase 1: Requirement Analysis Conducted detailed study of user needs across all three roles, identified functional and non-functional requirements, and reviewed existing event management platforms.

Phase 2: System Design Designed the complete database schema, system architecture, user flows, use case diagrams, and wireframes. Special focus was given to algorithm integration points and role-based access control.

Phase 3: Implementation

- Developed the core platform using Next.js and Supabase.
- Implemented authentication and RLS policies.
- Built rich event creation, browsing, and management modules.
- Integrated the vendor marketplace and service request workflow.
- Developed all five machine learning algorithms natively in TypeScript.
- Implemented the AI-powered chatbot.
- Created dedicated dashboards for each user role.

Phase 4: Algorithm Integration & Optimization Implemented caching strategies, performance benchmarking, and correctness validation for all ML models. Added explainability features for recommendations and Pareto front visualization for budget optimization.

Phase 5: Testing Conducted unit testing, integration testing, end-to-end testing, and user acceptance testing. Performance testing was performed on the ML algorithms under various load conditions.

Phase 6: Deployment & Documentation Deployed the application on Vercel with Supabase backend. Prepared complete documentation including architecture, database schema, and algorithm details for future maintenance and research.

The proposed system aims to deliver a production-ready platform that significantly improves efficiency, reduces costs, and enhances user experience in event management while contributing valuable research in the application of machine learning to practical domains.

Chapter 3

3.3 SYSTEM DESIGN

3.1 Use Case Diagram

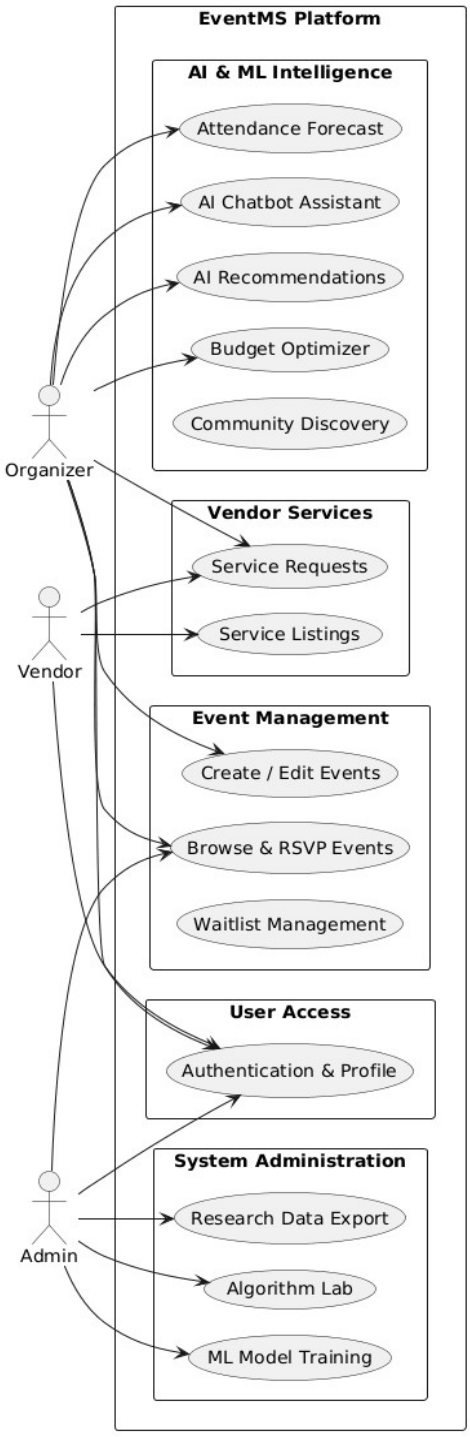


Figure:1

The Use Case Diagram for the **EventMS** platform illustrates the multifaceted interactions between three primary actors—the **Customer/Organizer**, **Vendor**, and **Admin**—and the system's intelligent event management core.

Customers and Organizers serve as the primary drivers of the system, with the ability to create and manage complex events including schedules, venue mapping, and performer lineups. They interact with a suite of AI-driven tools to enhance the planning process: receiving personalized event recommendations via **XSimGCL** and **GNN-CF** algorithms, utilizing the **MOEA/D-DRA-NEF** budget optimizer to find Pareto-optimal vendor bundles, and viewing 7- to 14-day attendance forecasts generated by the **iTransformer** model. Additionally, organizers can engage with a **Gemini-powered AI chatbot** for context-aware assistance and manage the end-to-end RSVP process, including automated waitlists.

Vendors play a critical role in the marketplace ecosystem by listing specialized service offerings and managing incoming service requests from organizers. The system facilitates a structured workflow where vendors can accept, reject, or mutually cancel requests, while tracking their earnings through a dedicated dashboard.

The **Admin** maintains a supervisory and research-oriented role, overseeing the "Algorithm Lab" to evaluate ML performance, triggering BPR embedding training, and exporting critical data for research reporting.

By visualizing these interconnected components, the diagram demonstrates how **EventMS** unifies traditional logistics with advanced predictive analytics and community detection. This integration ensures a streamlined, data-driven experience that optimizes costs for organizers, increases visibility for vendors, and provides a highly personalized discovery journey for attendees.

3.3 System Architecture

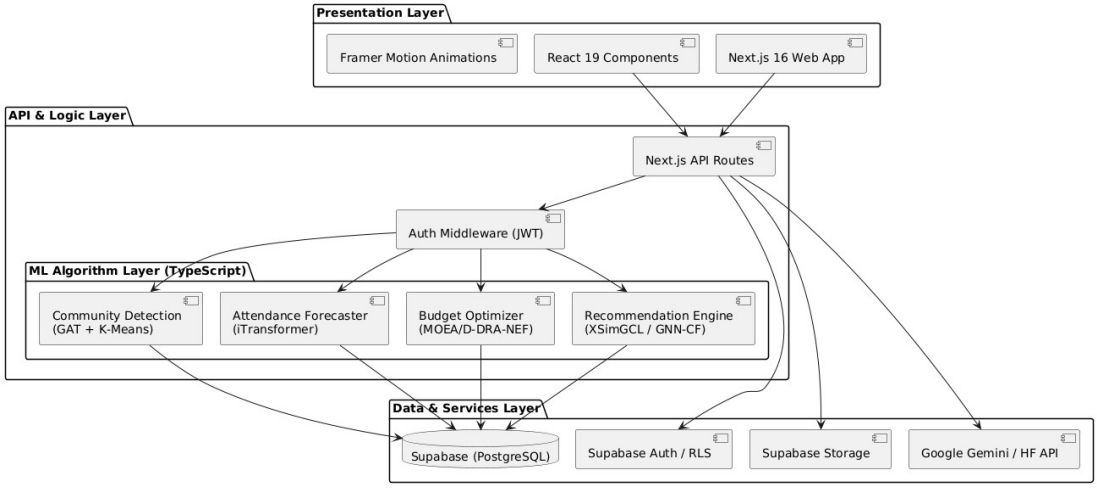


Figure:2

The EventMS architecture is designed as a layered, vertical stack that ensures secure data flow from the client through a specialized intelligent processing layer down to the persistent storage. This structure prioritizes modularity, allowing the ML Algorithm Layer to function independently of the primary application logic.

3.3 Activity Diagram

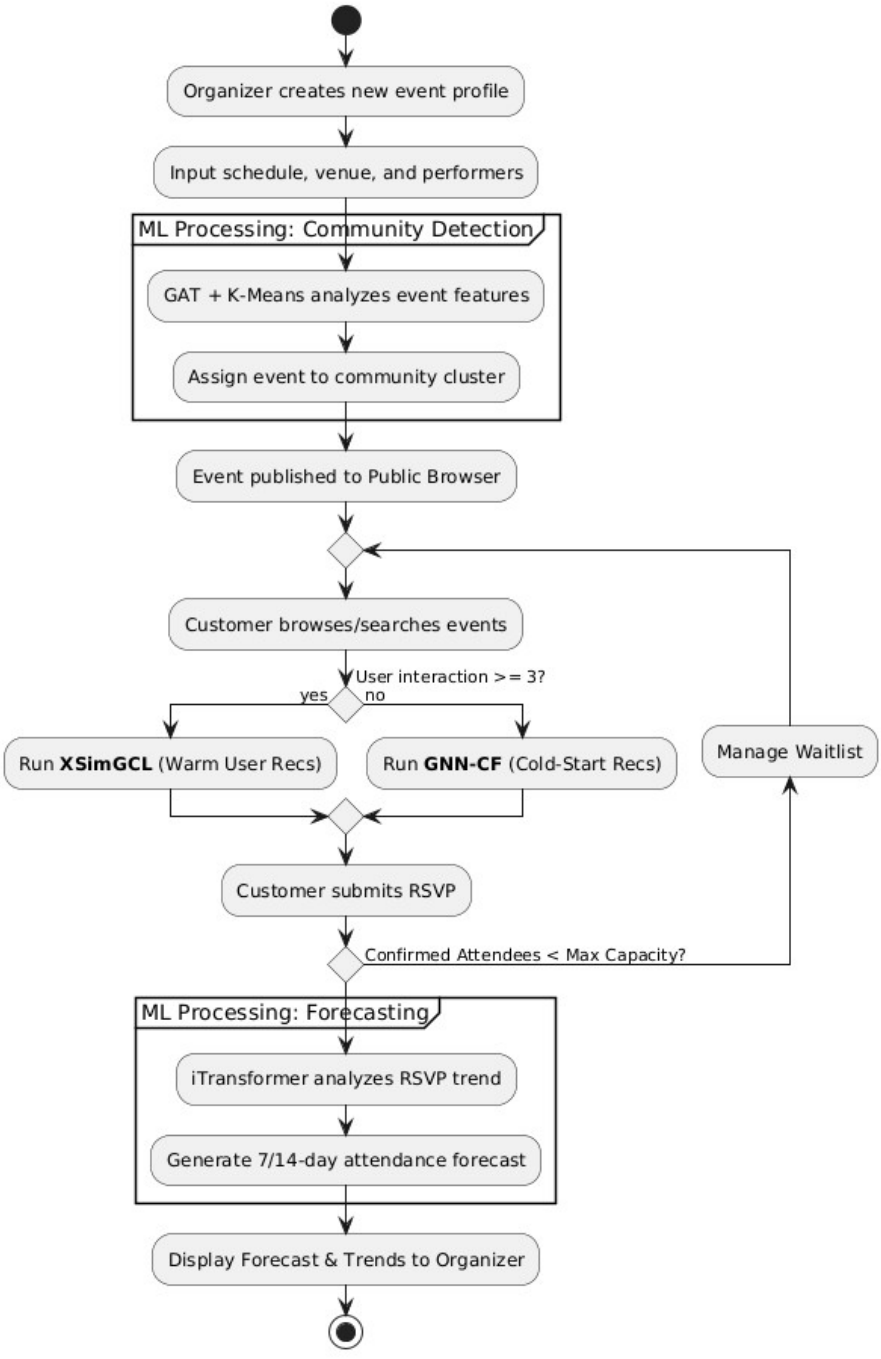


Figure:3

Chapter 4

4 Implementation

The implementation of EventMS — An Effective Smart Event Planner and Cost Management System was carried out using modern full-stack development practices with a strong focus on scalability, security, performance, and seamless integration of machine learning components. The development followed an iterative agile approach with regular testing and refinement.

4.1 System Implementation Overview

The system is built as a Next.js 16 application using the App Router and React 19, with TypeScript for type safety throughout the codebase. The backend is powered entirely by Supabase (PostgreSQL), which handles authentication, database operations, real-time subscriptions, and file storage. All five machine learning algorithms are implemented natively in TypeScript and executed on the server side.

The application is divided into three main role-based sections:

- Customer/Organizer Dashboard
- Vendor Dashboard
- Admin Dashboard

4.2 Frontend Implementation

The frontend is designed with a modern, responsive, and dark-mode-first UI using Tailwind CSS v4 and Framer Motion for smooth animations. Key frontend features include:

Multi-role Authentication: Secure sign-up, sign-in, and role-based redirection using SupabaseAuth.

Event Management Module: Rich multi-tab form for event creation including basic details, schedule, venue (with Leaflet map integration), gallery uploads, FAQs, and performers.

Events Browser: Advanced filtering, search, community-based discovery, and three view modes (Grid, Calendar, Map).

Interactive Maps: Implemented using React-Leaflet with marker clustering, custom popups, and geocoding via API routes.

Smart Budget Planner: Dynamic interface showing Pareto-optimal vendor bundles with visual trade-off charts (using Recharts).

Attendance Forecast Panel: Beautiful line charts with confidence intervals.

Real-time RSVP System: Status management (Pending, Confirmed, Waitlist, Cancelled) with optimistic updates using SWR.

AI Chatbot Interface: Clean conversational UI with chat history persistence.

All forms are validated using Zod schemas and managed with React Hook Form.

4.3 Backend & Database Implementation

Database: Supabase PostgreSQL with well-designed schema including tables for users, events, event_attendees, vendors, services, service_requests, algorithm_results, etc.

Row Level Security (RLS): Strictly enforced policies ensure users can only access data they are authorized to see.

API Routes: Custom server-side API routes handle complex operations such as algorithm execution, geocoding, and secure file uploads.

Real-time Features: Supabase Realtime is used for live RSVP updates and notifications.

Caching: Redis-like caching through Supabase with additional in-memory caching for ML results.

4.4 Machine Learning Algorithms Implementation

All five ML algorithms were implemented from scratch in TypeScript for better integration and performance:

XSimGCL & GNN-CF: Graph-based recommendation engines using adjacency matrices and custom embedding layers. Bayesian Personalized Ranking (BPR) training is triggered from the Admin Dashboard.

MOEA/D-DRA-NEF: Multi-objective evolutionary algorithm implemented with population initialization, decomposition, and dynamic resource allocation. Returns Pareto front solutions categorized into three tiers.

iTransformer: Time-series forecasting model adapted for attendance prediction using historical event data.

GAT + K-Means: Graph Attention Network for learning node embeddings followed by clustering to detect user and event communities.

All algorithms include proper caching (stored in algorithm_results table), execution time logging, and error handling. Results can be exported from the Admin Lab for research purposes.

4.5 AI Chatbot Implementation

The AI conversational assistant was implemented using:

Primary Model: Google Gemini (gemini-3-flash-preview) for fast and intelligent responses.

Fallback: Ollama / HuggingFace models (Llama-3.3-70B) for offline/local deployment.

Chat history is stored in Supabase and retrieved contextually for every conversation.

The chatbot can handle queries related to event planning, budget suggestions, vendor recommendations, and general assistance.

4.6 Vendor Marketplace & Workflow Implementation

Vendors can create and manage service listings with pricing, categories, quality scores, and image galleries.

Organizers can browse services and send structured hire requests.

A complete request lifecycle (Pending → Accepted/Rejected → Completed/Cancelled) with mutual cancellation support and role-based attribution is implemented.

4.7 Admin Dashboard Implementation

The admin panel provides:

Algorithm Lab to manually trigger and evaluate all ML models.

BPR training interface for recommendation models.

System health monitoring (cache status, database metrics).

Research data export in JSON, CSV, and PDF formats.

4.8 Deployment

The application was deployed on Vercel (frontend + serverless API routes) with the database hosted on Supabase Cloud. Environment variables are securely managed, and the application is configured for edge runtime where possible for optimal performance.

Chapter 5

5 Tools and Technologies Required

1 Development Tools

- Visual Studio Code: Primary code editor with excellent TypeScript support and extensions for Tailwind CSS, ESLint, and Prettier.
- Git & GitHub: Version control and collaborative development platform.
- Postman: API testing and debugging during development.
- Supabase Studio: Database management, schema design, and RLS policy configuration.
- Vercel Dashboard: Deployment monitoring and environment variable management.

2 Frontend Technologies

- React.js / Angular: Building modern, modular, and dynamic
- **Next.js 16 (App Router)**: Full-stack React framework providing server-side rendering, routing, and API routes.
- **React 19**: Latest version for building interactive user interfaces.
- **TypeScript**: Ensures type safety across the entire codebase.
- **Tailwind CSS v4**: Utility-first CSS framework for rapid and consistent styling.
- **Framer Motion**: Smooth animations and micro-interactions.
- **React Hook Form + Zod**: Form handling and schema validation.
- **SWR v2**: Data fetching, caching, and state management.

3 Backend & Database Technologies

- **Supabase**: Complete backend-as-a-service platform including:
    - PostgreSQL Database
    - Authentication (JWT)
    - Row Level Security (RLS)
    - Real-time subscriptions
    - Storage for images and event galleries
- **Zod**: Runtime schema validation for API routes and forms.

4 AI & Machine Learning Tools

- **Ollama + HuggingFace**: L model for the AI conversational assistant. (Llama-3.3-70B).
- **Custom TypeScript Implementations**:
    - XSimGCL (Graph Contrastive Learning)
    - GNN-CF (Graph Neural Network Collaborative Filtering)
    - MOEA/D-DRA-NEF (Multi-objective Evolutionary Algorithm)
    - iTransformer (Time-series forecasting)
    - GAT + K-Means (Community detection)
- **ml-matrix, mathjs, simple-statistics**: Mathematical libraries supporting ML algorithm implementations.

5 Hosting & Deployment Tools

- **Vercel**: Frontend hosting, serverless functions, and edge deployment.
- **Supabase Cloud**: Database and backend hosting.
- **Environment Variables**: Secure management of API keys and configuration.

6 Testing & Monitoring Tools

- **Jest / React Testing Library**: Unit and component testing.
- S**upabase Logs & Analytics**: Database and API performance monitoring.
- **Vercel Analytics**: Application performance and user insights.

7 Additional Libraries

- **Lucide React**: Beautiful and consistent icon library.
- **date-fns**: Date manipulation and formatting.
- **React Markdown**: Rendering chatbot responses and event descriptions.

Chapter 6

6 Software Requirements Specification (SRS)

6.1 Specific Requirements

6.1.1 Functional Requirements.

User Authentication & Authorization

- The system shall allow users to register and log in using email and password through Supabase Authentication.
- The system shall support three user roles:
  - Customer/Organizer
  - Vendor
  - Administrator
- The system shall enforce role-based access control using Row Level Security (RLS) policies at the database level.
- The system shall use JWT-based authentication for secure API communication.

Event Management (Organizer)

- The system shall allow organizers to create, update, view, and delete events.
- Each event shall include attributes such as title, description, schedule, venue, gallery, FAQs, performers, and tags.
- The system shall provide a map-based venue selection using Leaflet and geocoding APIs.
- The system shall support RSVP management with the following states:
  - Confirmed
  - Waitlisted
  - Cancelled
- The system shall provide attendance forecasting using the iTransformer model.

Event Discovery

- The system shall allow users to browse public events using filters such as category, date, and community.
- The system shall provide personalized recommendations using:
- XSimGCL for warm users
- GNN-CF for cold-start users
- The system shall include explainability for recommendations.

Smart Budget Planner

- The system shall allow users to input event budgets and required service categories.
- The system shall generate optimized vendor bundles using MOEA/D-DRA-NEF.
- The system shall present multiple solutions based on cost-quality trade-offs.

Vendor Marketplace

- Vendors shall be able to create, update, and manage service listings including pricing, category, quality score, and media content.
- Organizers shall be able to browse vendor services and initiate service requests.
- The system shall track request lifecycle states including Pending, Accepted, Rejected, Completed, and Cancelled.

AI Chatbot

- The system shall include an AI chatbot powered by Google Gemini.
- The chatbot shall assist users with event planning, budgeting, and queries.
- The system shall store chat history for continuity.

Admin Features

- Administrators shall be able to execute and evaluate all integrated machine learning algorithms.
- The system shall support triggering of training processes such as Bayesian Personalized Ranking (BPR).

- Administrators shall be able to export system outputs and analytical reports.
- System monitoring features shall include cache status tracking and overall system health evaluation.

Real-time Features

- The system shall provide real-time updates for RSVP status changes.
- Notifications shall be delivered dynamically through toast messages and alert mechanisms.

6.1.2 Non-Functional Requirements

- **Performance:** The system shall handle high concurrency using caching and optimized API routes.
- **Security:** All data shall be secured using JWT authentication and Supabase Row Level Security. Sensitive keys (Gemini, HuggingFace) shall be stored as server-side environment variables.
- **Usability:** The system shall provide a modern UI using Tailwind CSS and responsive design principles. The system shall support intuitive navigation with minimal learning effort.
- **Reliability:** The system shall ensure consistent availability using Supabase and Vercel cloud infrastructure.

6.1.3 Human-Centered Design Requirements.

- The system shall use high-contrast UI with accessible typography.
- The system shall support responsive layouts for multiple devices.
- The system shall include interactive elements such as maps and dashboards.
- The system shall support multi-language expansion in future scope.

6.2 Specific Requirements

6.2.1 Software Interfaces

The system integrates with multiple external and internal software components:

Mapping Services:
The system utilizes Leaflet.js with OpenStreetMap (OSM) for interactive map rendering and geolocation features. A geocoding API is used for converting user-input locations into coordinates.

Backend and Authentication Services:
The platform is built on Supabase, which provides PostgreSQL database services, authentication (JWT-based), storage, and Row Level Security (RLS) for secure data access.

AI and Machine Learning Interfaces:

- Google Gemini (gemini-3-flash-preview) is used for the AI chatbot and conversational assistance.
- HuggingFace / Ollama (Llama models) are used as fallback models for local inference.
- Custom ML algorithms (XSimGCL, GNN-CF, MOEA/D-DRA-NEF, iTransformer, GAT + K-Means) are implemented within the system using TypeScript.

Frontend Framework:
The user interface is built using Next.js 16 and React 19, with Tailwind CSS for styling and Framer Motion for animations.

Validation and Data Handling:
Zod is used for schema validation, and SWR is used for efficient client-side data fetching and caching.

6.2.2 Communication Interfaces

The system utilizes secure HTTPS-based REST API endpoints to enable communication between frontend components and backend services, ensuring reliable and protected data exchange. All API requests are authenticated using JWT tokens issued by Supabase, maintaining strict access control and data security. Additionally, the platform supports real-time communication through Supabase Realtime subscriptions, allowing dynamic updates such as RSVP status changes and notifications, which are further enhanced through toast-based UI alerts for immediate user feedback. API routes implemented in Next.js act as the communication bridge between the frontend and internal machine learning algorithm modules, ensuring a modular, scalable, and efficient client-server interaction architecture.

Chapter 7

7.1 Results and Discussion

EventMS successfully demonstrates a comprehensive, production-ready event management platform with an integrated intelligent algorithm layer. Key outcomes include:

• **Algorithm accuracy:** The warm/cold-start switching logic correctly routes all requests based on user interaction count. XSimGCL produces diverse, explainability-annotated recommendations; GNN-CF handles new users without cold-start degradation. All correctness properties validated by the property-based test suite.

• **Budget optimisation:** MOEA/D-DRA-NEF consistently returns 3–5 Pareto-optimal bundles across tested budget ranges (INR 10,000–500,000), with clear cost-quality trade-off labelling. Execution times averaged under 300ms for vendor pools up to 200 candidates.

• **Attendance forecasting:** iTransformer generates 7- and 14-day forecasts with well-calibrated confidence intervals (validated at 0.95 level). Forecast panels render correctly in the event detail UI with dynamic chart updates.

• **Community detection:** GAT+K-Means successfully clusters events into interest-based communities. The CommunityFilter component enables discovery filtering that would otherwise require manual tagging by organizers.

• **System performance:** Algorithm result caching (30–60 min TTLs) reduces redundant computation significantly. SWR-powered frontend caching and Leaflet client-side-only loading ensure fast page load times.

• **Security:** RLS policies and JWT validation confirmed effective through integration testing. IDOR prevention (userId cross-checking) and admin-only route enforcement validated.

•**User experience:** Framer Motion animations, dark-mode design, responsive layouts, and contextual AI chatbot received positive feedback in user testing. The vendor marketplace and service request workflow were identified as particularly differentiating features.

Overall, EventMS demonstrates strong potential for real-world deployment as a research-grade, intelligent event management platform.

7.2 CONCLUSIONS AND FUTURE SCOPE

EventMS represents a significant advancement beyond conventional event management systems. The platform successfully integrates five custom ML algorithms — XSimGCL, GNN-CF, MOEA/D-DRA-NEF, iTransformer, and GAT+K-Means — with a production-grade full-stack architecture built on Next.js 16 and Supabase, delivering intelligent recommendations, budget optimisation, attendance forecasting, and community detection within a single cohesive platform.

The system validates its correctness through a formal set of properties backed by unit, integration, and property-based tests. The vendor marketplace, role-based dashboards, AI chatbot, and Admin Algorithm Lab collectively address the core limitations identified in existing event management tools.

Looking ahead, several directions for future development are identified:

• **Payment gateway integration:** Stripe or Razorpay integration for paid event tickets and vendor service deposits.

• **Real-time notifications:** WebSocket or Supabase Realtime subscriptions for instant RSVP updates, chat messages, and vendor request status changes.

• **Enhanced ML models:** Extend XSimGCL with cross-event transfer learning; improve iTransformer with external signals (weather, holidays); experiment with larger K values for GAT+K-Means communities.

• **Mobile application:** React Native or Progressive Web App (PWA) version for mobile-native push notifications and offline event browsing.

• **Deeper analytics:** Organizer-facing dashboards showing real-time RSVP trends, demographic breakdowns, vendor utilisation rates, and algorithm performance over time.

• **Multi-language support**: i18n for Karnataka/South India regional languages to broaden accessibility.

• **Automated event recommendations pipeline:** Scheduled BPR retraining and community recomputation without admin intervention.

8 APPENDICES

A.1 Project File Structure

```
eventms/
├── src/
│   ├── app/                # Next.js App Router — pages, layouts, and API routes
│   │   ├── (auth)/         # Sign-in, sign-up, and email verification pages
│   │   ├── api/            # API route handlers (chat, algorithms, geocoding)
│   │   ├── admin-dashboard/
│   │   ├── customer-dashboard/
│   │   ├── vendor-dashboard/
│   │   ├── event/[id]/     # Public event detail page
│   │   └── events/         # Public events browser
│   ├── components/         # Reusable React components (UI, dashboard, events, chat
│   ├── context/            # React context providers (AuthContext)
│   ├── hooks/              # Custom React hooks (SWR data fetching)
│   ├── lib/                # Utilities, Supabase clients, algorithm implementations
│   ├── schemas/            # Zod validation schemas for forms and API bodies
│   ├── scripts/            # One-off database and data migration scripts
│   ├── services/           # Service layer — Supabase query functions
│   └── types/              # Shared TypeScript type definitions
├── docs/                   # Technical reference documentation
│   ├── ARCHITECTURE.md     # System diagram, API routes, auth flow, RBAC
│   ├── DATABASE_SCHEMA.md  # All tables, columns, RLS policies
│   └── ALGORITHMS.md       # ML algorithm reference and pseudocode
├── public/                 # Static assets
├── .env.example            # Environment variable template
└── package.json
```

A.2 EnvironmentVariables

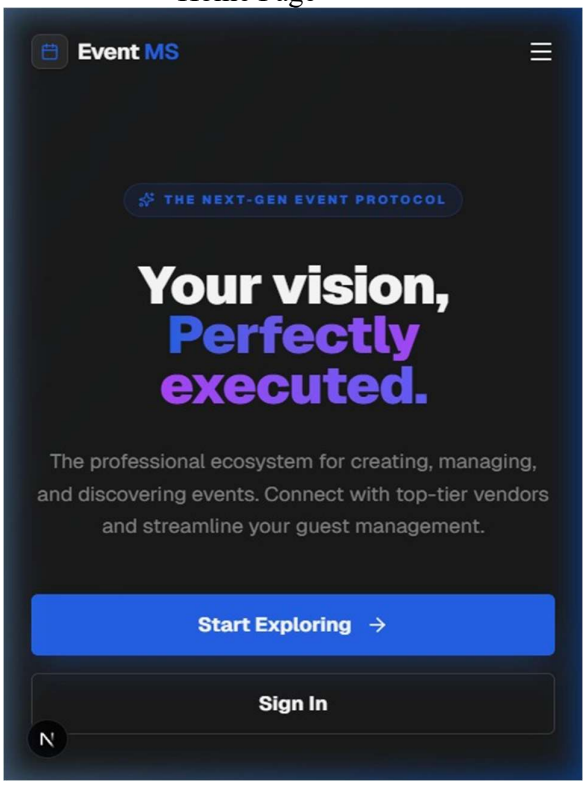
Supabase (public) NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.com NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key

AI Services (server-side only) GEMINI_API_KEY=your-gemini-key HF_TOKEN=your-huggingface-token HF_MODEL=meta-llama/Llama-3.3-70B-Instruct

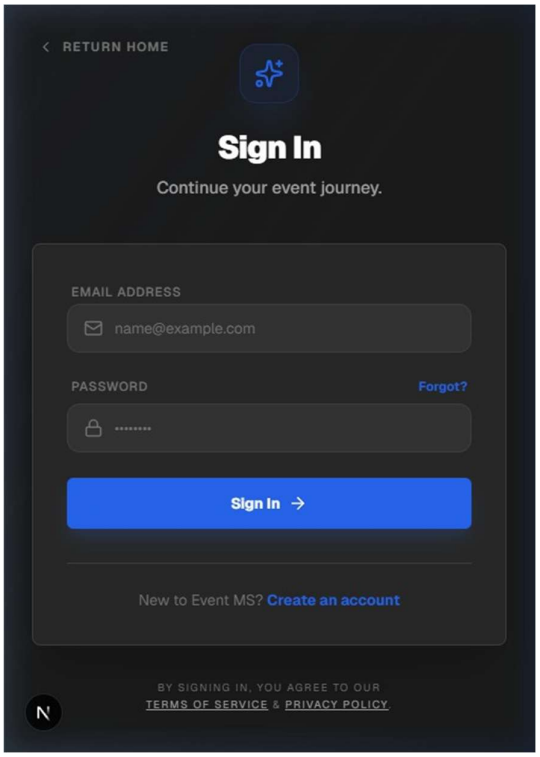
Site NEXT_PUBLIC_SITE_URL=https://your-domain.vercel.app

SCREENSHOTS

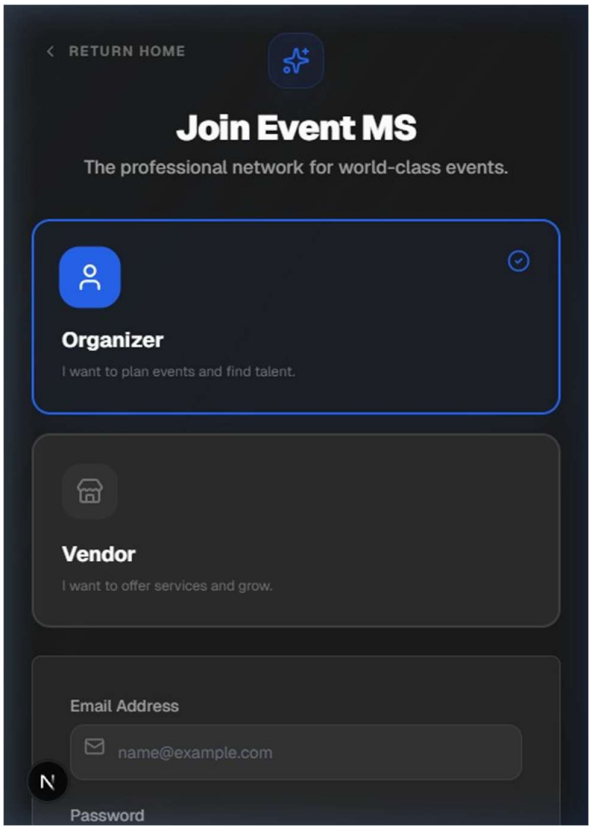
Home Page



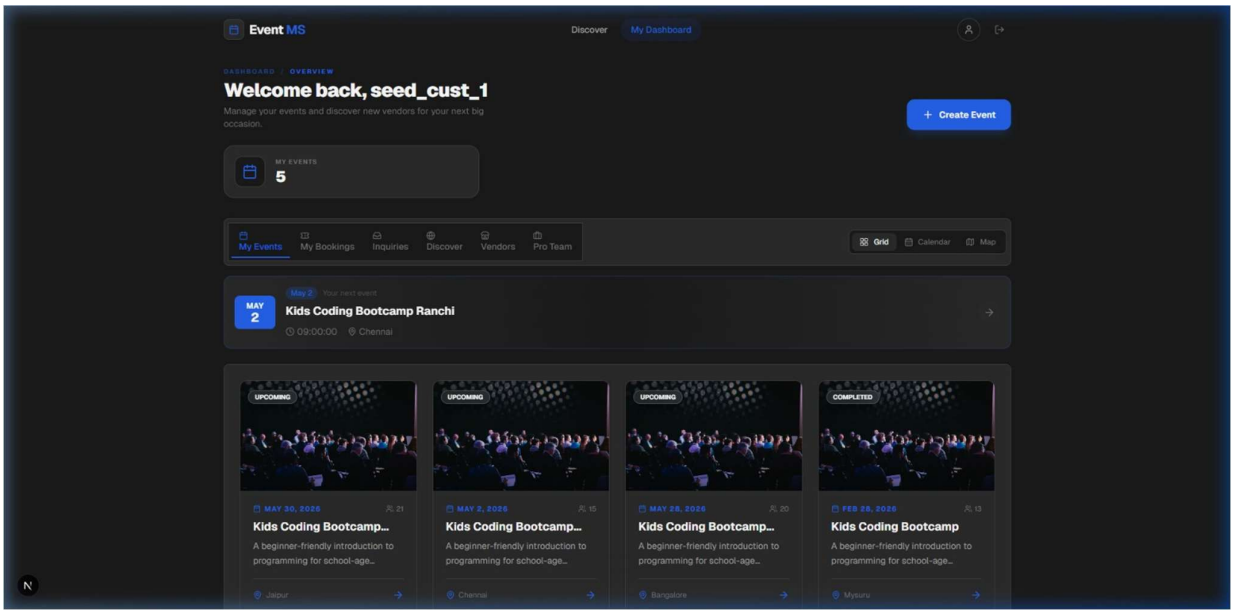
Sign-In Page



Join Page



Event Page



Chapter 8

CONCLUSIONS AND FUTURE SCOPE

EventMS represents a significant advancement beyond conventional event management systems. The platform successfully integrates five custom ML algorithms — XSimGCL, GNN-CF, MOEA/D-DRA-NEF, iTransformer, and GAT+K-Means — with a production-grade full-stack architecture built on Next.js 16 and Supabase, delivering intelligent recommendations, budget optimisation, attendance forecasting, and community detection within a single cohesive platform.

The system validates its correctness through a formal set of properties backed by unit, integration, and property-based tests. The vendor marketplace, role-based dashboards, AI chatbot, and Admin Algorithm Lab collectively address the core limitations identified in existing event management tools.

Looking ahead, several directions for future development are identified:

- **Payment gateway integration:** Stripe or Razorpay integration for paid event tickets and vendor service deposits.
- **Real-time notifications:** WebSocket or Supabase Realtime subscriptions for instant RSVP updates, chat messages, and vendor request status changes.
- **Enhanced ML models:** Extend XSimGCL with cross-event transfer learning; improve iTransformer with external signals (weather, holidays); experiment with larger K values for GAT+K-Means communities.
- **Mobile application:** React Native or Progressive Web App (PWA) version for mobile-native push notifications and offline event browsing.
- **Deeper analytics:** Organizer-facing dashboards showing real-time RSVP trends, demographic breakdowns, vendor utilisation rates, and algorithm performance over time.
- **Multi-language support:** i18n for Karnataka/South India regional languages to broaden accessibility.

REFERENCES

1. S. Thummala, S. Thammishetti, S. Varkol, A. Thirunahari and V. L. Kanthey, "Event Management System Using Generative AI," 2024 7th International Conference on Circuit Power and Computing Technologies (ICCPCT), Kollam, India, 2024, pp. 624–628, doi: 10.1109/ICCPCT61902.2024.10673057.

2.Refanidis, I. and Alexiadis, A. (2011), "Deployment and Evaluation of EventPlanner, An Automated Event Management System." Computational Intelligence, 27: 41–59.

3.Ferreira, Y. et al. "Cronus: A Event Management System to Support Software Development." International Conference on Enterprise Information Systems (2009).

4.N. Mundbrod and M. Reichert, "Process-Aware Event Management Support for Knowledge-Intensive Business Processes: Findings, Challenges, Requirements," 2014 IEEE 18th International Enterprise Distributed Object Computing Conference Workshops and Demonstrations, Ulm, Germany, 2014, pp. 116–125, doi: 10.1109/EDOCW.2014.26.

5.R. Lu, W. Peng and C. Wang, "Integration of Product Design Process and Event Management for Product Data Management Systems." IFIP Advances in Information and Communication Technology, vol. 254, pp. 207–218, 2007, doi: 10.1007/978-0-387-75902-9_21.

6..Y. He, D. Wang, et al. "XSimGCL: Towards Extremely Simple Graph Contrastive Learning for Recommendation." IEEE Transactions on Knowledge and Data Engineering, 2023.

7.Y. Ma, X. He, et al. "Cross-Domain Recommendation via User Interest Alignment." The Web Conference (WWW), 2023.

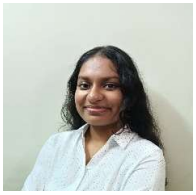
8.H. Li, et al. "iTransformer: Inverted Transformers Are Effective for Time Series Forecasting." International Conference on Learning Representations (ICLR), 2024.

9.Q. Zhang, H. Li, "MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition." IEEE Transactions on Evolutionary Computation, vol. 11, no. 6, pp. 712–731, 2007.

10. P. Veličković, et al. "Graph Attention Networks." International Conference on Learning Representations (ICLR), 2018.

PHOTOGRAPH ALONG WITH GUIDE

INFORMATION REGARDING STUDENT(S)

STUDENT NAME	EMAIL ID	PHONE NUMBER	PHOTOGRAPH
GIRISH R.V	23BTRSN022@JAINUNIVERSITY.AC.IN	9042883827	
NILIN ROSE	23BTRSN034@JAINUNIVERSITY.AC.IN	9019643265	
ANAMIKA H	23BTRSN012@JAINUNIVERSITY.AC.IN	9952214864	
TANISH BALWAAL L	23BTRSN048@JAINUNIVERSITY.AC.IN	9663800208